Algorithmes et Pensée Computationnelle

Consolidation POO + Classes abstraites et interfaces

Les exercices sont construits autour des concepts d'héritage, de classes abstraites et d'interfaces. Au terme de cette séance, vous serez en mesure de différencier une classe abstraite d'une interface, de savoir à quel moment utiliser l'une ou l'autre et de factoriser votre code afin de le rendre mieux structuré et plus lisible.

Cette feuille d'exercices est divisée en trois sections. La première porte sur des notions de base en POO, la deuxième sur les classes abstraites et la dernière sur les interfaces.

Les exercices de cette séance seront réalisés en Java. Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `code`.

1 Consolidation - Programmation Orientée Objet

Question 1: (*10 minutes*) Encapsulation - Java
L'encapsulation sert à cacher les détails d'implémentation. Elle permet de ne montrer aux utilisateurs que les informations essentielles.

En Java, il est recommandé de déclarer les attributs comme étant `private` et de mettre à disposition des utilisateurs des méthodes publiques d'accès afin qu'ils puissent accéder à la valeur des attributs privés ou la modifier.

Le nom des méthodes d'accès commence par `get` ou `set`, suivi du nom de l'attribut dont la première lettre est mise en majuscule (Java Naming convention).

Le mot-clé `this` en Java est une variable de référence qui fait référence à l'objet actuel. Il est utilisé dans une méthode d'instance ou un constructeur pour accéder aux membres de l'objet actuel, tels que les variables d'instance, les méthodes et les constructeurs.

1. Dans votre IDE, créez une classe `Person`,
2. Ajoutez-y un attribut privé `name`,
3. Créez un getter et un setter pour l'attribut `name` en suivant la convention de nommage des méthodes.
4. Dans votre `main`, créez une instance de `Person`,
5. En utilisant le setter défini précédemment, donnez un nom (`name`) à votre instance.
6. Affichez le nom de l'instance en utilisant le getter de l'attribut `name`.

Solution

```
1
2  public class Person {
3      private String name;
4
5      public String getName() {
6          return name;
7      }
8
9      public void setName(String newName) {
10         this.name = newName;
11     }
12
13     public static void main(String[] args) {
14         Person myObj = new Person();
15         myObj.setName("John");
16         System.out.println(myObj.getName());
17     }
```

```
18  }
```

Question 2: (*10 minutes*) Héritage - Java
Comme vous le savez, il est possible que des classes-filles héritent des attributs ou méthodes de classes-mères. En Java, il faut utiliser le mot-clé `extends` lorsqu'on définit une classe-fille. Ainsi, l'héritage permet la réutilisation des attributs et méthodes d'une classe existante.

1. Créez une classe-mère `Vehicle` ayant un attribut protégé (`protected`) de type chaîne de caractères appelé `brand`.
2. Créez un constructeur pour la classe `Vehicle` et assignez la valeur passée comme argument à l'attribut `brand`.
3. Définissez une méthode `honk` qui affiche `"Tuut, tuut!"`
4. Créez une classe-fille `Car` qui hérite de `Vehicle` et possède un attribut `modelName` avec valeur par défaut `"Mustang"`.

Conseil

Dans le constructeur de la classe-fille, n'oubliez pas de faire appel au constructeur de la classe-mère en utilisant le mot-clé `super`.

Solution

```
1
2  public class Vehicle {
3      protected String brand;
4
5      public Vehicle(String brand) {
6          this.brand = brand;
7      }
8
9      public void honk() {
10         System.out.println("Tuut, tuut!");
11     }
12
13     public static void main(String[] args) {
14         Car myCar = new Car("Ford");
15         myCar.honk();
16         System.out.println(myCar.brand + " " + myCar.modelName);
17     }
18 }
```

Solution

```
1  class Car extends Vehicle {
2
3      public String modelName = "Mustang";
4
5      public Car(String brand) {
6          super(brand);
7      }
8  }
```

Question 3: (*30 minutes*) [Java] Redéfinition des méthodes d'égalité et d'identité
En Java, créez une classe `Fraction` ayant pour attributs privés un numérateur et un dénominateur.

Conseil

Vous pouvez vous inspirer des solutions d'exercices de la semaine 8.

Assignez des valeurs à vos attributs dans le constructeur que vous définirez. Dans le corps de votre classe, effectuez les opérations suivantes :

- Définir une méthode nommée `gcd` qui prend comme arguments les deux attributs définis précédemment et qui retourne leur plus grand diviseur commun. Pour cela, vous devez implémenter l'algorithme d'Euclide de façon récursive.

  Conseil

  L'algorithme d'Euclide fonctionne comme suit :
  - Si a = 0 alors gcd(a, b) = b.
  - Si b = 0 alors gcd(a, b) = a.
  - Si aucune des conditions ci-dessus n'est remplie, faire appel à `gcd` de façon récursive avec pour premier argument `b` et pour deuxième `a%b`.

- Définir une méthode `simplify` qui sera appelée dans le constructeur après l'initialisation des attributs.
- Redéfinir la méthode `toString` pour produire une représentation textuelle des instances de `Fraction`.
- Redéfinir la méthode `equals` pour tester l'égalité entre deux fractions. Cette méthode prendra comme argument un objet `other` de type `Object`.

  Conseil

  En Java, par défaut toutes les classes héritent d'une classe-mère nommée `Object`. Vous pouvez utiliser le même test d'égalité que vous avez fait pour les exercices de la semaine 7.

- Pour tester l'égalité dans la méthode `equals` :
  - Vérifier si `this` et `other` sont le même objet. Si oui, renvoyer `true`.
  - Vérifier si `other` est `null` ou si `this` et `other` sont de types différents. Si oui, renvoyer `false`.
  - Sinon, vérifier si les deux fractions sont numériquement égales (même si leurs numérateurs et dénominateurs sont différents) et renvoyer le résultat de cette comparaison.

  Conseil

  En Java, pour vérifier que deux objets sont de même type, vous pouvez accéder à la classe de chacun des éléments en utilisant la méthode `.getClass()` et les comparer. Une fois que vous avez vérifié le type de l'objet `other` passé comme argument, le convertir en `Fraction` (utiliser type casting de la semaine 3).

- Dans le `main`, exécuter le code suivant (disponible sur Moodle) :

```
1 public class Question3 {
2     public static void main(String[] args) {
3         Fraction f1 = new Fraction(8, 32);
4         Fraction f2 = new Fraction(1, 4);
5         System.out.println(f1 == f2);
6         System.out.println(f1.equals(f2));
7     }
8 }
```

.

- Pourquoi les lignes 5 et 6 renvoient-elles des résultats différents ?

Solution

```java
public class Fraction {
    private int numerateur;
    private int denominateur;

    public Fraction(int numerateur, int denominateur) {
        this.numerateur = numerateur;
        this.denominateur = denominateur;
        simplify();
    }

    public int gcd(int a, int b) {
        if (a == 0) {
            return b;
        } else if (b == 0) {
            return a;
        } else {
            return gcd(b, a % b);
        }
    }

    private void simplify() {
        int pgcd = gcd(this.numerateur, this.denominateur);
        this.numerateur = this.numerateur / pgcd;
        this.denominateur = this.denominateur / pgcd;
    }

    @Override
    public String toString() {
        return numerateur + "/" + denominateur;
    }

    @Override
    public boolean equals(Object other) {
        if (this == other)
            return true;
        if (other == null || this.getClass() != other.getClass())
            return false;
        Fraction f = (Fraction) other;
        return this.numerateur * f.denominateur == f.numerateur *
    this.denominateur;
    }
}
```

L'instruction `f1 == f2` vérifie que les deux objets sont identiques. Dans notre cas, `f1` et `f2` sont deux objets différents. On peut le vérifier en comparant leur représentation numérique. Cela peut être fait en utilisant la méthode `hashCode()`. Vous pouvez essayer en exécutant la ligne suivante dans le `main` : `System.out.println(f1.hashCode())`.
L'instruction `f1.equals(f2)` fait appel à la méthode `equals()` de notre classe `Fraction` et exécute les instructions que nous avons définies dans cette méthode. L'instruction == compare les références, alors que `.equals()` compare les valeurs.

2 Classes abstraites

Question 4: (*15 minutes*) Création d'une classe abstraite
Une classe abstraite sert de base à des sous-classes et ne peut pas être instanciée directement. Elle se déclare avec le mot-clé `abstract` et peut contenir des méthodes concrètes (avec une implémentation) ainsi que des méthodes abstraites (sans implémentation).

Une sous-classe qui n'implémente pas toutes les méthodes abstraites dont elle hérite doit être déclarée abstraite.

```
// Exemple de classe abstraite
public abstract class Animal {
    protected int speed;

    // Déclaration d une méthode abstraite
    abstract void run();
}
```

- Créez une classe abstraite appelée `Item` avec quatre attributs d'instance et un attribut de classe, qui sont les suivants :

```
private static int count = 0; // Attribut de classe
private String name;
private int id;
private double price;
private List<String> ingredients;
```

> **Conseil**
>
> Les attributs d'instance sont créés lors de l'instanciation d'un objet (à l'aide du mot-clé `new`) et détruits lors de la destruction de l'objet. Les attributs de classes (attributs statiques), quant à eux, sont créés lors de l'exécution du programme et détruits lors de l'arrêt du programme. En Java, les attributs de classe sont accessibles en utilisant le nom de la classe soit : `ClassName.AttributeName`.

- Créez un constructeur pour initialiser les attributs `name, price, ingredients et id`. L'attribut `id` sera incrémenté à chaque instanciation de la classe.

> **Conseil**
>
> Pensez à utiliser `count` pour initialiser la valeur d'`id`. Ainsi, dans le constructeur, `id` sera égal à `++count`.

- Implémentez les `getters` des attributs `id, name, price et ingredients`.
- Implémentez la méthode `equals(Object o)`.
- Redéfinissez `toString()` pour renvoyer une chaîne de quatre lignes au format suivant, en remplaçant les valeurs entre accolades par celles de l'objet :

```
ID: {id}
Name: {name}
Price: {price} CHF
List of ingredients: {ingredients}
```

La liste des ingrédients est entourée de crochets et ses éléments sont séparés par une virgule suivie d'une espace, par exemple `[rice, tomato, onion]`.

> **Conseil**
>
> La méthode `equals` permet de comparer deux objets. Elle prend en paramètre un objet de type `Object` et renvoie `true` si l'objet actuel est égal à l'objet passé en paramètre.

Solution

```
import java.util.*;

public abstract class Item {

    private int id;
```

```
6       private static int count = 0;
7       private String name;
8       private double price;
9       private List<String> ingredients;
10
11      public Item(String name, double price, List<String> ingredients) {
12          this.id = ++count;
13          this.name = name;
14          this.price = price;
15          this.ingredients = ingredients;
16      }
17
18      public int getId() {
19          return this.id;
20      }
21
22      public String getName() {
23          return this.name;
24      }
25
26      public double getPrice() {
27          return this.price;
28      }
29
30      public List<String> getIngredients() {
31          return this.ingredients;
32      }
33
34      @Override
35      public boolean equals(Object o) {
36          if (o instanceof Item) {
37              Item i = (Item) o;
38              return i.getId() == this.getId();
39          }
40          return false;
41      }
42
43      @Override
44      public String toString() {
45          return "ID: " + this.id +
46                  "\nName: " + this.name +
47                  "\nPrice: " + this.price + " CHF" +
48                  "\nList of ingredients: " + this.ingredients;
49      }
50  }
```

La variable `count` est un attribut de classe (indiqué par `static`), partagé par toutes les instances de la classe. Comme le constructeur l'incrémente à chaque appel, il compte le nombre d'instances créées. En écrivant `this.id = ++count;`, chaque instance reçoit donc un `id` unique.

Question 5: (🕓 *15 minutes*) Classe abstraite et types d'attributs

- Implémentez une classe abstraite `Figure` contenant deux attributs protégés de type `float` : `largeur` et `longueur`, un constructeur qui initialise ces deux attributs, et deux méthodes abstraites : `getAire`() et `getPerimetre`() qui renvoient un `float`.
- Créez deux classes `Carre` et `Rectangle` qui héritent de la classe `Figure`. Le constructeur de `Rectangle` prend en paramètre une largeur et une longueur, celui de `Carre` ne prend que la longueur du côté. À l'intérieur de ces classes, implémentez les méthodes `getAire()` et `getPerimetre()`.
- Dans le `main` d'une classe `FigureMain`, créez un carré et un rectangle et affichez le périmètre du carré et l'aire du rectangle.

Conseil

Un attribut protégé est accessible aussi bien dans la classe-mère que dans la classe-fille. On utilise le mot-clé `protected` pour rendre des attributs protégés.
Pour rappel, pour créer une classe fille, on utilise le mot-clé `extends`. Par exemple : `public class Carre extends Figure`.

Solution

```java
public abstract class Figure {

    protected float largeur;
    protected float longueur;

    public Figure(float largeur, float longueur) {
        this.largeur = largeur;
        this.longueur = longueur;
    }

    public abstract float getPerimetre();

    public abstract float getAire();
}
```

Solution

```java
class Carre extends Figure {

    public Carre(float largeur) {
        super(largeur, largeur);
    }

    @Override
    public float getPerimetre() {
        return this.largeur * 4;
    }

    @Override
    public float getAire() {
        return this.largeur * this.largeur;
    }

}
```

Solution

```java
class Rectangle extends Figure {

    public Rectangle(float largeur, float longueur) {
        super(largeur, longueur);
    }

    @Override
    public float getPerimetre() {
        return (this.largeur + this.longueur) * 2;

    }
```

```
12
13      @Override
14      public float getAire() {
15          return this.largeur * this.longueur;
16      }
17  }
```

Solution

```
1   class FigureMain {
2       public static void main(String[] args) {
3           Carre c = new Carre(5.0f);
4           Rectangle r = new Rectangle(4.0f, 3.0f);
5           System.out.println(c.getPerimetre());
6           System.out.println(r.getAire());
7       }
8   }
```

3 Interfaces

Question 6: (*15 minutes*) Interface et héritage (Liée à la question 4)

En Java, une interface se déclare comme suit :

```
1  public interface IMakeSound {
2      double MY_DECIBEL_VALUE = 75;
3
4      void makeSound();
5  }
```

Pour rappel : dans une interface Java, les variables sont toujours `public static final`, et les méthodes sont publiques et abstraites par défaut.
Les méthodes déclarées dans une interface doivent être implémentées dans des sous-classes :

```
1
2  public class Cat extends Animal implements IMakeSound {
3      public void makeSound() {
4          System.out.println("I meow at" + MY_DECIBEL_VALUE + "decibel.");
5      }
6
7      // Implémentation d une méthode abstraite
8      void run() {
9          this.speed += 10;
10     }
11 }
```

- Créez une interface `Edible` contenant une méthode `eatMe` qui ne retourne aucune valeur.
- Créez une interface `Drinkable` contenant une méthode `drinkMe` qui ne retourne aucune valeur.
- Créez une classe `Food` qui hérite de la classe `Item` (définie à la question 4) et qui implémente l'interface `Edible`. Écrivez le constructeur de `Food` et la méthode `eatMe` (dans la classe `Food`).

Conseil

Vous pouvez reprendre la classe `Item` de la question 4.
Dans la méthode `eatMe()`, vous pouvez simplement afficher un message en utilisant un `println`.

Certains aliments ne sont pas seulement `Edible` (mangeables) mais aussi `Drinkable` (buvables) comme les soupes par exemple.

- Créez une classe `Soup` qui hérite de `Food` et implémente l'interface `Drinkable`. Ensuite, écrivez le constructeur de la classe `Soup` et la méthode `drinkMe` (dans la classe `Soup`).

Vous pouvez ensuite tester les méthodes `eatMe()` et `drinkMe()` à l'aide des lignes suivantes, qui créent des instances de `Soup` et `Food`.

```
1 Soup s1 = new Soup("Kizili soup", 7.7, new ArrayList<String>(List.of("bulgur",
      "meat", "tomato")));
2 Food f = new Food("Stuffed peppers", 12, new ArrayList<String>(List.of("rice",
      "tomato", "onion")));
3 s1.eatMe();
4 s1.drinkMe();
5 f.eatMe();
```

Solution

```
1  public interface Drinkable {
2      void drinkMe();
3  }
```

Solution

```
1  public interface Edible {
2      void eatMe();
3  }
```

Solution

```
1  import java.util.*;
2
3  public class Food extends Item implements Edible {
4      public Food(String name, double price, List<String> ingredients) {
5          super(name, price, ingredients);
6      }
7
8      public void eatMe() {
9          System.out.println("Eat me!" + toString());
10     }
11 }
```

Solution

```
1  import java.util.*;
2
3  class Soup extends Food implements Drinkable {
4      public Soup(String name, double price, List<String> ingredients) {
5          super(name, price, ingredients);
6      }
7
8      public void drinkMe() {
9          System.out.println("Drink the soup !" + this.toString());
10         // ou System.out.println("Drink the soup !" + this);
11     }
12 }
```